

Part 10: Jumps

1. **What is the difference between conditional and unconditional jumps?** Unconditional jumps (`JMP`) always happen. Conditional jumps (like `JE`) only happen if specific Flags (Zero, Carry, etc.) are set.
2. **Which instruction is used for unconditional jump?** The `JMP` instruction.
3. **Name conditional jump instructions.** `JE` (Equal), `JNE` (Not Equal), `JG` (Greater), `JL` (Less), `JC` (Carry), `JZ` (Zero), `JS` (Sign).
4. **How does `JE/JZ` work?** It jumps if the Zero Flag (ZF) is 1, meaning the previous comparison found the two values were equal.
5. **How does `JNE/JNZ` work?** It jumps if the Zero Flag (ZF) is 0, meaning the values were not equal.
6. **How does `JC` work?** It jumps if the Carry Flag (CF) is 1. Used to handle unsigned overflow or borrows.
7. **How does `JNC` work?** It jumps if the Carry Flag (CF) is 0 (No Carry).
8. **How does `JL/JNGE` work?** It jumps if the destination was **Less** than the source (Signed comparison, checks Sign and Overflow flags).
9. **How does `JG/JNLE` work?** It jumps if the destination was **Greater** than the source (Signed comparison).
10. **How does `JO/JNO` work?** It jumps if the Overflow Flag (OF) is set (`JO`) or not set (`JNO`). Used to detect signed arithmetic errors.
11. **How do jumps work with loops?** They are used to create custom loops (Jump back to start) or to break out of loops early (Jump to end).
12. **How do you implement a simple decision using jump?** Use `CMP` to compare values. If the condition is true, jump to the "True" block; otherwise, let the code fall through to the "False" block.
13. **How do you implement a nested if using jumps?** If the first check passes, jump to a label that contains the second `CMP` and jump. If any check fails, jump to the end.
14. **Which flag registers are checked during jumps?** The Status Flags: Zero (ZF), Carry (CF), Sign (SF), Overflow (OF), and Parity (PF).
15. **How do jumps affect program flow?** They change the value of the **IP** (Instruction Pointer) register, causing the CPU to fetch the next instruction from a different memory address.